FitBuddy-AI Fitness Plan Generator using Gemini Models

Phase 6 – Project Testing

1. Introduction

Testing is performed to verify that FitBuddy behaves as expected and to identify problems before final demonstration. Testing covers the application startup process, routes, forms, AI integration, database operations, feedback workflow, administration, and health monitoring.

2. Environment and Startup Testing

The first test is to activate the project virtual environment and start the application with Uvicorn. The expected result is that the server starts successfully and reports that the application startup is complete.

During development, dependency errors were encountered when packages such as SQLAlchemy, python-dotenv, Jinja2, the Google GenAI SDK, and python-multipart were not available in the active environment. These errors demonstrated the importance of installing project dependencies inside the same virtual environment used to start Uvicorn.

3. Route Testing

The home route is tested by opening the root URL. The expected result is the FitBuddy input page. The generate-workout route is tested by submitting valid form data. The expected result is an AI-generated plan followed by a result page.

The feedback route is tested with an existing user. The expected result is a revised plan. The view-all-users route is tested with both a valid and invalid

admin key. The delete route is checked to ensure that only authorized deletion is allowed. The health route is checked for a successful status response.

4. Database Testing

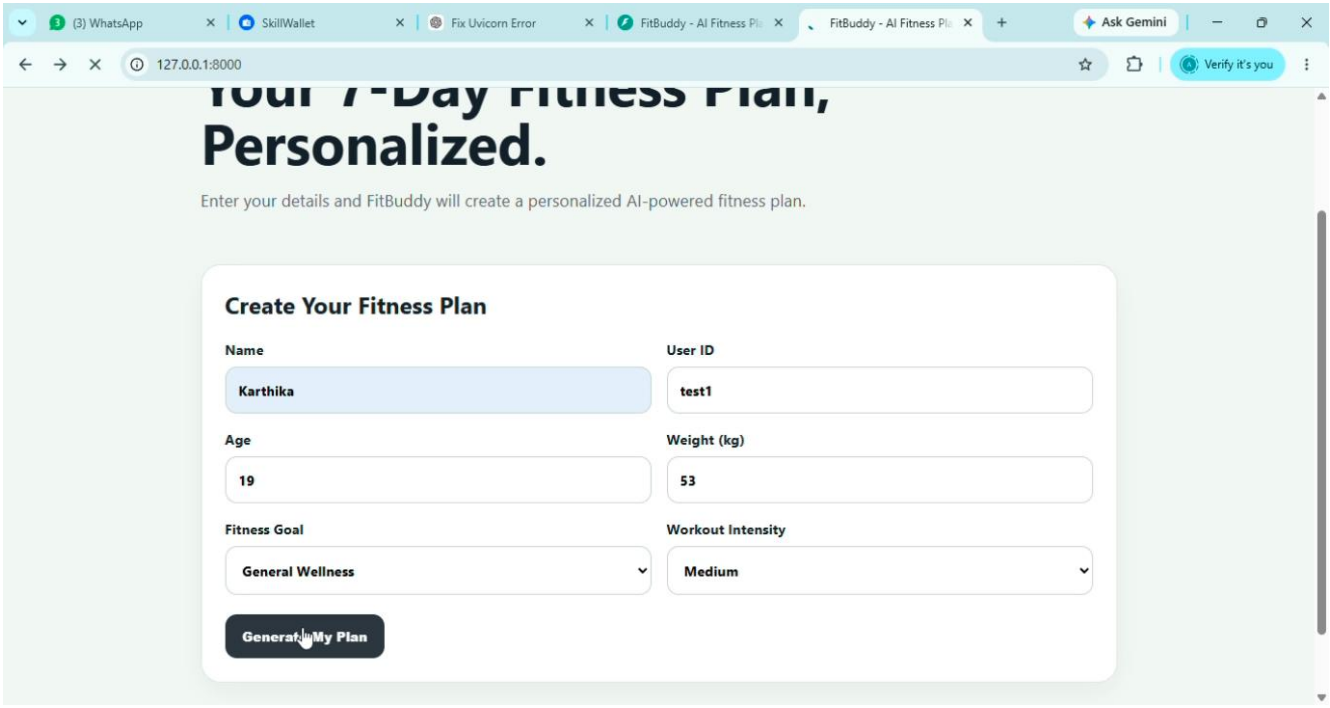
Database testing verifies that a new user is inserted correctly and that the stored fields match the submitted information. A second submission using the same user ID is used to verify the existing-user update logic.

The feedback test verifies that updated_plan and feedback are stored. The user-list test verifies that stored users can be retrieved. The deletion test confirms that the selected record is removed.

5. AI Testing

AI testing checks whether the Gemini workout generator can be imported and called correctly and whether the nutrition generator produces a usable response. Because AI generation depends on an external service, API availability, credentials, model availability, and quota can affect the result.

The application should therefore handle generation exceptions and display an error rather than crashing without explanation.



6. Template and Static Resource Testing

The template configuration must point to the correct templates directory. The required files index.html, result.html, and all_users.html must exist. The static directory must also exist if StaticFiles is configured.

During development, a missing static directory caused application startup to fail. A separate issue occurred when the template response was incorrectly configured, producing an error during page rendering. These issues show why template paths and TemplateResponse arguments must be checked carefully.

7. Sample Test Cases

Test Case 1: Start server — Expected: Uvicorn starts and application startup completes.

Test Case 2: Open home — Expected: index.html loads successfully.

Test Case 3: Submit valid workout form — Expected: workout generation begins and result page is displayed.

Test Case 4: Submit feedback for an existing user — Expected: revised plan is generated and stored.

Test Case 5: Use invalid admin key — Expected: access is rejected.

Test Case 6: Delete authorized user — Expected: selected user is removed.

Test Case 7: Open health endpoint — Expected: service status is returned.

Test Case 8: Submit invalid or incomplete form data — Expected: validation/error response instead of silent failure.

8. Conclusion

Testing is essential for verifying both individual features and the complete integrated workflow. The development errors observed during implementation were useful because they identified missing dependencies, incorrect imports,

missing resources, and configuration problems. After these issues are corrected, the project can be demonstrated with greater confidence.

Phase Deliverables / Evidence

Item	Purpose	Status / Evidence
Startup tests	Checks environment/server	Tested during development
Endpoint tests	Checks routes and validation	Tested during development
Integration tests	Checks AI/database/workflow	Tested during development